Haoquan Wang
u1570975
2026-4-8

Assignment 10: BinaryMaxHeap

1. My programming partner is Jake Scott and I submitted the program to Gradescope.

2. N is the total number of elements in the list to be sorted. δ (delta) is the maximum distance any element can be from its correctly sorted position — in other words, each element in a δ-sorted list is within δ positions of where it belongs in the fully sorted result.

   The expected time complexity of the DeltaSorter algorithm is O(N log δ). This is because for each of the N elements, we perform one add and one extractMax on a BinaryMaxHeap that holds at most δ + 1 elements. Both heap operations cost O(log δ), giving a total of O(N log δ).

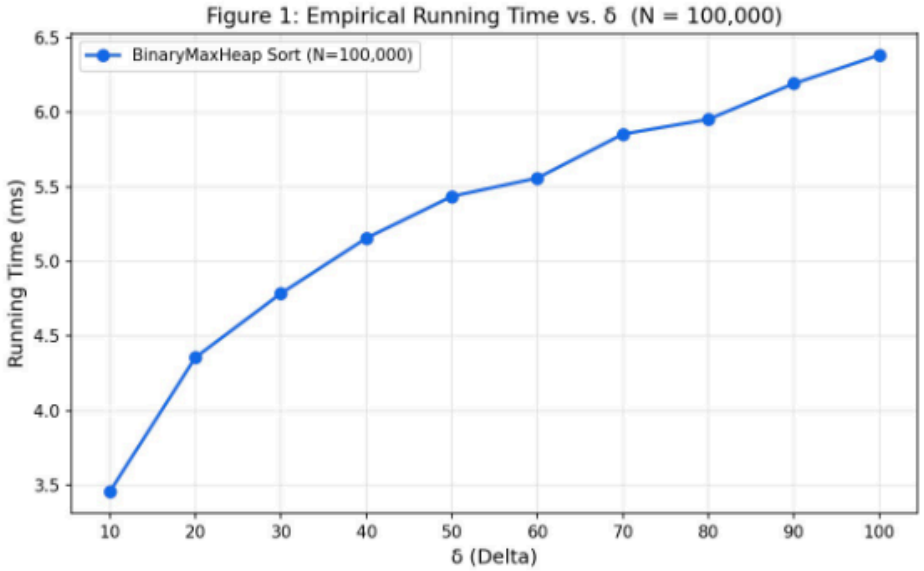
   Since δ is a small constant, log δ is also a small constant. The algorithm reduces to O(N) behavior in practice — the heap stays very small and both add and extractMax are nearly O(1). The algorithm is extremely efficient.

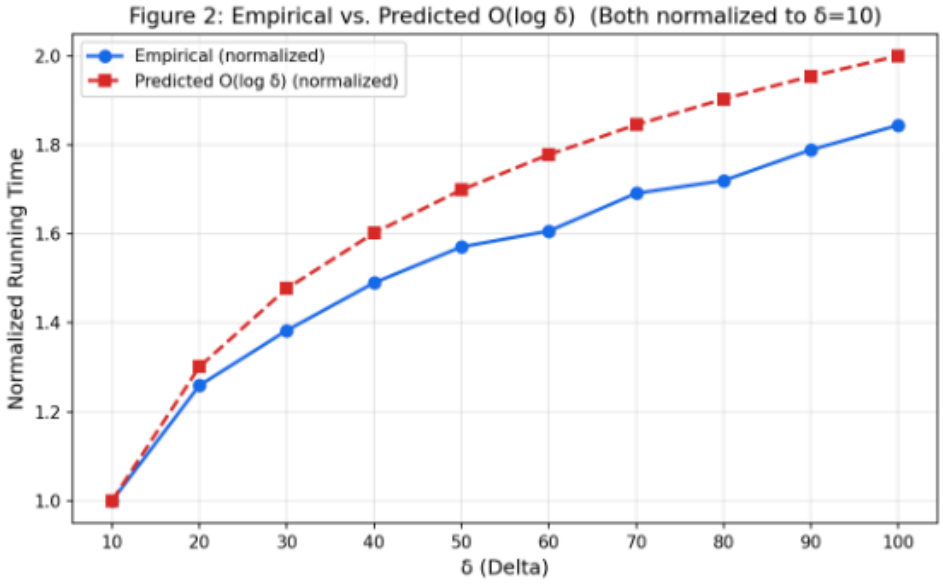
   Now δ grows with N, so log δ ≈ log(N/10) ≈ log N. The complexity becomes O(N log N), similar to general-purpose sorting algorithms. The heap can hold up to N/10 + 1 elements, making each operation more expensive.

   The algorithm uses a BinaryMaxHeap that holds at most δ + 1 elements at any time. Therefore, the extra space required is O(δ). When δ is small, this is nearly O(1); when δ = N/10, the extra space is O(N).

3.

   3.1. A list of 100,000 integers was generated using generateDescendingDeltaSortedArrayList for δ values ranging from 10 to 100 in steps of 10. DeltaSorter.sort was called for each δ, and the median of 50 runs was recorded.





3.2. The heap used by DeltaSorter always holds at most δ + 1 elements. Each add and extractMax operation costs O(log(δ + 1)). Since N is fixed at 100,000, the only variable affecting running time is the height of the heap, which is log δ. Therefore, as δ doubles, the running time increases by only one additional level of the heap — a logarithmic relationship.

3.3. To verify the O(log δ) prediction, both the empirical times and the predicted $\log_2(\delta)$ curve were normalized to their value at δ = 10. If the behavior is truly O(log δ), the two normalized curves should closely overlap.The empirical ratios are slightly below the predicted values, likely due to CPU caching effects and constant-factor overhead that diminish at larger δ. Overall, the trend confirms the O(log δ) behavior.

4.

4.1. When δ is small and fixed, insertion sort benefits greatly from the near-sorted structure of the input. Each element needs to travel at most δ positions to reach its sorted location, so the inner loop runs at most δ times per element. The total number of comparisons is O(N · δ) = O(N · 10) = O(N). Insertion sort requires O(1) extra space — it sorts in-place.

4.2. Merge sort does not exploit the δ-sorted structure. It always divides and merges regardless of input order, giving O(N log N) time for all cases. Merge sort requires O(N) extra space for the temporary arrays used during the merge step.

Figure 3: Empirical Running Time – Three Strategies (δ = 10)

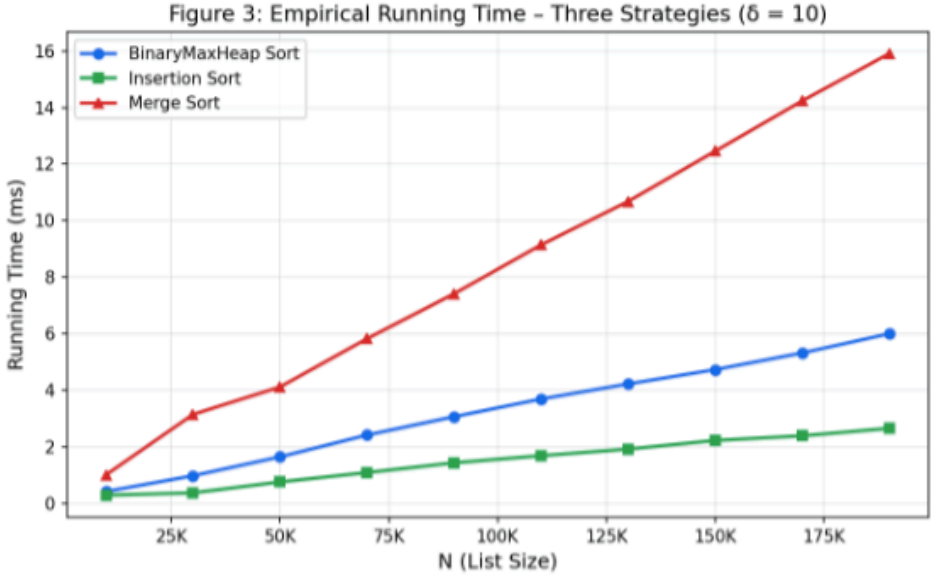
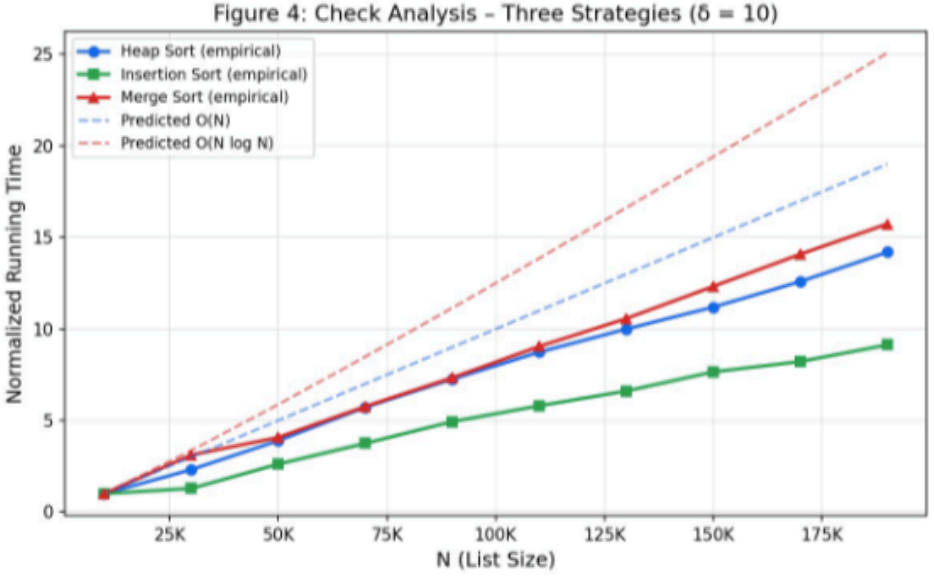


Figure 4: Check Analysis – Three Strategies (δ = 10)



4.3. Both the empirical data and the predicted curves were normalized to each strategy's time at N = 10,000.

BinaryMaxHeap Sort: The empirical curve closely tracks the O(N) predicted line. Since δ = 10 is constant, log δ is constant, and the algorithm runs in linear time. This matches our observation — the heap sort line is nearly perfectly linear.

Insertion Sort: The empirical curve also closely tracks O(N). Because δ = 10 bounds the inner loop, insertion sort is also linear here. Notably, insertion sort is the fastest of the three strategies when δ is small — it has low constant overhead and excellent cache behavior on nearly-sorted data.

Merge Sort: The empirical curve tracks O(N log N) as expected. Merge sort is consistently the slowest for δ = 10 because it cannot exploit the sorted structure and always allocates extra arrays at each recursive level.

5.

5.1. Expected Big-O: Insertion Sort (δ = N/10)

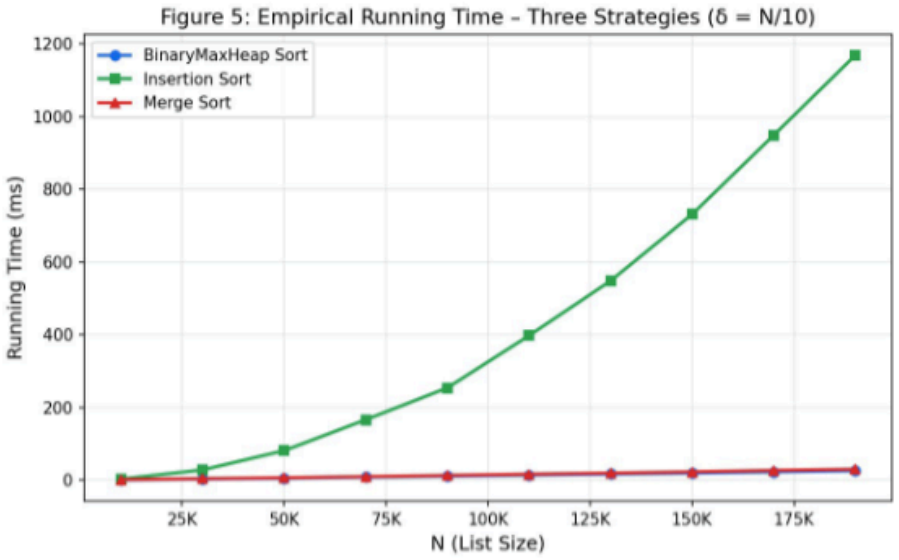
With δ = N/10, each element may need to travel up to N/10 positions. The total work becomes $O(N \cdot \delta) = O(N \cdot N/10) = O(N^2/10) = O(N^2)$. Insertion sort degrades to quadratic behavior and requires O(1) extra space.

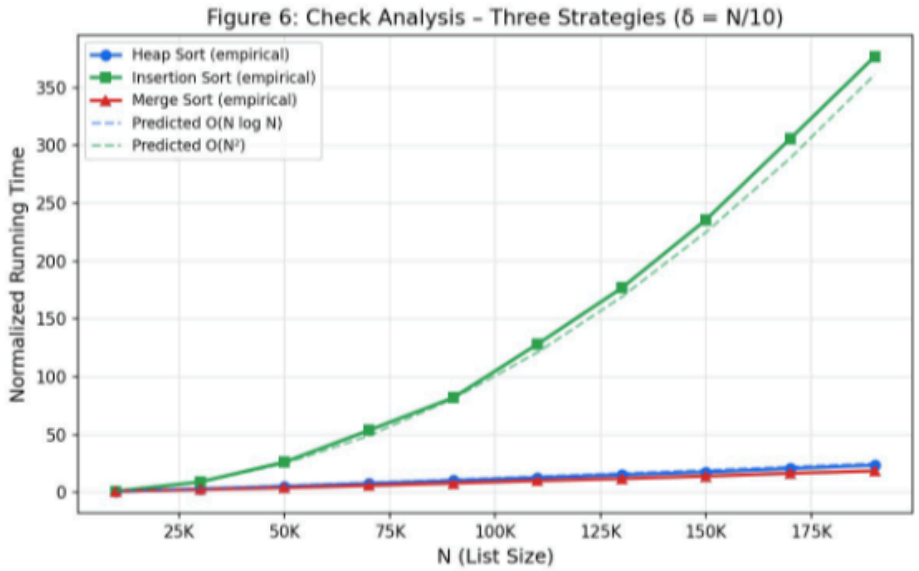
5.2. Expected Big-O: Merge Sort (δ = N/10)

Merge sort is unaffected by δ — it always runs in O(N log N) time regardless of the input's sorted state. Extra space is O(N).

5.3. Expected Big-O: BinaryMaxHeap Sort (δ = N/10)

With δ = N/10, the heap holds up to N/10 + 1 elements. Each operation costs $O(\log(N/10)) = O(\log N - \log 10) = O(\log N)$. Total: O(N log N). Extra space is $O(\delta) = O(N)$.





5.4. BinaryMaxHeap Sort: The empirical curve closely matches O(N log N). The slight upward curve compared to perfect linear growth is consistent with the growing log factor as N increases.

Insertion Sort: The empirical curve grows far faster than the others, closely matching the predicted $O(N^2)$ curve. At N = 190,000, insertion sort took over 1,168 ms — roughly 45× slower than heap sort at the same size. This confirms the quadratic degradation when δ is proportional to N.

Merge Sort: The empirical curve tracks O(N log N) closely, consistent with the expectation. It is slightly faster than heap sort in practice, likely due to better cache locality in the merge step compared to heap operations.

6. Without knowing δ in advance, the best strategy is BinaryMaxHeap Sort. Here is the reasoning:

    6.1. When δ is small (e.g., δ = 10), heap sort runs in O(N log δ) ≈ O(N) time, nearly as fast as insertion sort and far faster than merge sort.
    6.2. When δ is large (e.g., δ = N/10), heap sort still runs in O(N log N), matching merge sort and vastly outperforming insertion sort's O(N²).
    6.3. Insertion sort is the fastest when δ is tiny, but degrades catastrophically as δ grows — making it risky without knowing δ.
    6.4. Merge sort is consistently O(N log N) but cannot adapt to small δ and always uses O(N) extra space.
    6.5. Heap sort uses only O(δ) extra space — when δ is small this is nearly O(1), a significant advantage over merge sort.

    In summary, BinaryMaxHeap Sort provides the best worst-case guarantee O(N log N), the best best-case behavior O(N) when δ is small, and uses less extra space than merge sort. It is the most robust and adaptive choice when δ is unknown.